

INSTITUTE OF ENGINEERING
ADVANCED COLLEGE OF ENGINEERING AND MANAGEMENT
KALANKI, KATHMANDU
(AFFILIATED TO TRIBHUVAN UNIVERSITY)



ADVANCED COLLEGE
OF ENGINEERING AND MANAGEMENT

LAB REPORT
SUBJECT: Distributed System
LAB NO: 3

Submitted By:

Name: Himal Joshi
Roll No: ACE079BCT028
Date: July 22, 2026

Submitted To:

**Department of Electronics and
Computer Engineering**

Implementation of DFS (Distributed File System)

Objective

The main objectives of this laboratory assignment are:

1. To implement a Distributed File System (DFS) using a client-server architecture in Python.
2. To perform fundamental file operations remotely, including CREATE, READ, WRITE, and DELETE.
3. To utilize standard Python networking (socket) and concurrency (threading) libraries.
4. To ensure transparent file access without requiring manual network connection prompts from the user.

Theory

DFS (Distributed File System)

A Distributed File System (DFS) is a network-based file management system that allows users to access and manipulate files stored on a remote server as if they were residing on their local storage devices. A DFS provides location transparency, centralized data storage, concurrent multi-user access control, and data consistency across distributed clients over a network.

The architecture relies on the classical Client-Server paradigm:

- **Server:** Operates continuously on a designated network interface and port. It receives structured file operation requests over TCP sockets, processes them against physical server storage (server_storage/), and sends back JSON-serialized responses. Multi-threading is utilized to handle multiple concurrent client connections.
- **Client:** Connects automatically to the server's default host and port address. It presents an interactive terminal interface for the user to select file operations, serializes requests into JSON payloads, sends them to the server, and renders the server's response.

Block Diagram

The structural block diagram of the implemented Distributed File System is shown in Figure 1.

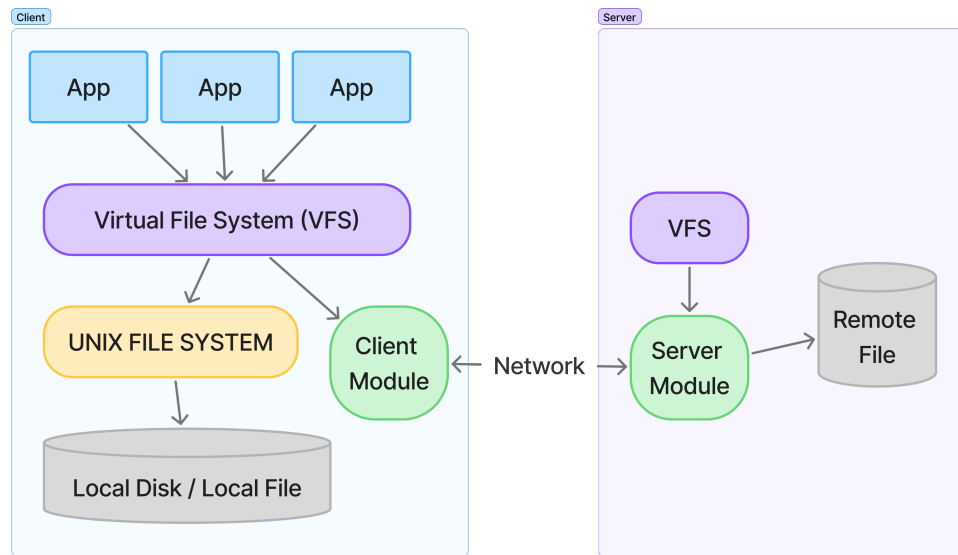


Figure 1: Implementation of DFS - Block Diagram

Directory Service, Flat File Service

In a formal DFS model, services are categorized into two primary abstractions:

1. **Directory Service:** Manages file names, directory structures, path resolution, and metadata mapping. In this system, the directory service maps text filenames to localized paths in `server_storage/` and provides directory listings.
2. **Flat File Service:** Handles raw byte-stream operations on file contents. It handles file creation, reading data blocks, writing/updating file contents, and deleting file entities from disk storage.

Source Code and Output

server.py

Source Code

```
1  import socket
2  import os
3  import json
4  import threading
5
6  # Server Configuration
7  HOST = '127.0.0.1'
8  PORT = 5000
9  STORAGE_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)),
    ↪ 'server_storage')
10
11 # Ensure server storage directory exists
12 if not os.path.exists(STORAGE_DIR):
13     os.makedirs(STORAGE_DIR)
14
15 def handle_client(client_socket, address):
16     print(f"[+] Client connected from {address[0]}:{address[1]}")
17     while True:
18         try:
19             # Receive request data from client
20             raw_data = client_socket.recv(4096).decode('utf-8')
21             if not raw_data:
22                 break
23
24             request = json.loads(raw_data)
25             action = request.get("action", "").upper()
26             filename = request.get("filename", "")
27             content = request.get("content", "")
28
29             response = {"status": "ERROR", "message": "Invalid Action", "data":
    ↪ None}
30
31             # Sanitize filename to prevent directory traversal
32             safe_filename = os.path.basename(filename) if filename else ""
33             filepath = os.path.join(STORAGE_DIR, safe_filename) if
    ↪ safe_filename else ""
34
35             # 1. CREATE File
36             if action == "CREATE":
37                 if not safe_filename:
38                     response = {"status": "ERROR", "message": "Filename
    ↪ required."}
```

```

39         elif os.path.exists(filepath):
40             response = {"status": "ERROR", "message": f"File
↳ '{safe_filename}' already exists."}
41         else:
42             with open(filepath, 'w', encoding='utf-8') as f:
43                 f.write(content)
44             response = {"status": "SUCCESS", "message": f"File
↳ '{safe_filename}' created successfully."}
45
46     # 2. READ File
47     elif action == "READ":
48         if not os.path.exists(filepath):
49             response = {"status": "ERROR", "message": f"File
↳ '{safe_filename}' not found."}
50         else:
51             with open(filepath, 'r', encoding='utf-8') as f:
52                 file_data = f.read()
53             response = {"status": "SUCCESS", "message": f"File
↳ '{safe_filename}' read successfully.", "data":
↳ file_data}
54
55     # 3. WRITE File (Update / Overwrite)
56     elif action == "WRITE":
57         if not safe_filename:
58             response = {"status": "ERROR", "message": "Filename
↳ required."}
59         else:
60             with open(filepath, 'w', encoding='utf-8') as f:
61                 f.write(content)
62             response = {"status": "SUCCESS", "message": f"File
↳ '{safe_filename}' written successfully."}
63
64     # 4. DELETE File
65     elif action == "DELETE":
66         if not os.path.exists(filepath):
67             response = {"status": "ERROR", "message": f"File
↳ '{safe_filename}' not found."}
68         else:
69             os.remove(filepath)
70             response = {"status": "SUCCESS", "message": f"File
↳ '{safe_filename}' deleted successfully."}
71
72     # 5. LIST Files
73     elif action == "LIST":
74         files = os.listdir(STORAGE_DIR)
75         response = {"status": "SUCCESS", "message": "Files listed
↳ successfully.", "data": files}

```

```

76
77     # Send JSON response back to client
78     client_socket.sendall(json.dumps(response).encode('utf-8'))
79
80     except json.JSONDecodeError:
81         err_resp = {"status": "ERROR", "message": "Invalid JSON payload."}
82         client_socket.sendall(json.dumps(err_resp).encode('utf-8'))
83     except Exception as e:
84         err_resp = {"status": "ERROR", "message": str(e)}
85         client_socket.sendall(json.dumps(err_resp).encode('utf-8'))
86         break
87
88     print(f"[-] Client {address[0]}:{address[1]} disconnected.")
89     client_socket.close()
90
91 def start_server():
92     server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
93     # Reuse address option to allow immediate restart of server
94     server_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
95
96     server_socket.bind((HOST, PORT))
97     server_socket.listen(5)
98     print(f"=====")
99     print(f" Distributed File System (DFS) Server")
100    print(f" Running on {HOST}:{PORT}")
101    print(f" Storage Path: {STORAGE_DIR}")
102    print(f"=====")
103    print("Waiting for client connections...\n")
104
105    try:
106        while True:
107            client_socket, address = server_socket.accept()
108            # Handle client requests in a separate thread
109            client_thread = threading.Thread(target=handle_client,
110            ↪ args=(client_socket, address))
111            client_thread.daemon = True
112            client_thread.start()
113        except KeyboardInterrupt:
114            print("\n[!] Shutting down DFS Server...")
115        finally:
116            server_socket.close()
117
118 if __name__ == "__main__":
119     start_server()

```

Output

```
=====
Distributed File System (DFS) Server
Running on 127.0.0.1:5000
Storage Path: e:\Lab Reports\Distributed-System-Lab\LAB-3-dfs\server_storage
=====
Waiting for client connections...
```

client.py

Source Code

```
1  import socket
2  import json
3
4  # Server Connection Details (hardcoded as per requirements)
5  HOST = '127.0.0.1'
6  PORT = 5000
7
8  def send_request(action, filename="", content=""):
9      """
10     Sends a request dictionary to the DFS Server and receives the response.
11     """
12     try:
13         # Create TCP Socket
14         client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
15         client_socket.connect((HOST, PORT))
16
17         payload = {
18             "action": action,
19             "filename": filename,
20             "content": content
21         }
22
23         # Send JSON payload
24         client_socket.sendall(json.dumps(payload).encode('utf-8'))
25
26         # Receive JSON response
27         raw_response = client_socket.recv(4096).decode('utf-8')
28         response = json.loads(raw_response)
29
30         client_socket.close()
31         return response
32
33     except ConnectionRefusedError:
34         return {"status": "ERROR", "message": "Could not connect to DFS Server.
        ↳ Is server.py running?"}
```

```

35     except Exception as e:
36         return {"status": "ERROR", "message": f"Connection error: {str(e)}"}
37
38 def main():
39     while True:
40         print("\n=====")
41         print("    DISTRIBUTED FILE SYSTEM (DFS) CLIENT    ")
42         print("=====")
43         print("1. Create File")
44         print("2. Read File")
45         print("3. Write / Update File")
46         print("4. Delete File")
47         print("5. List Files")
48         print("6. Exit")
49         print("=====")
50
51         choice = input("Enter option (1-6): ").strip()
52
53         # 1. CREATE File
54         if choice == '1':
55             filename = input("Enter filename to create (e.g., test.txt): ")
56             ↵ ").strip()
57             if not filename:
58                 print("[!] Filename cannot be empty.")
59                 continue
60             content = input("Enter initial file content (optional): ")
61             response = send_request("CREATE", filename, content)
62             print(f"\n[Server Status] : {response.get('status')}")
63             print(f"[Server Message]: {response.get('message')}")
64
65         # 2. READ File
66         elif choice == '2':
67             filename = input("Enter filename to read: ").strip()
68             if not filename:
69                 print("[!] Filename cannot be empty.")
70                 continue
71             response = send_request("READ", filename)
72             print(f"\n[Server Status] : {response.get('status')}")
73             print(f"[Server Message]: {response.get('message')}")
74             if response.get("status") == "SUCCESS":
75                 print("-----")
76                 print("FILE CONTENT:")
77                 print(response.get("data", ""))
78                 print("-----")
79
80         # 3. WRITE / Update File
81         elif choice == '3':
82             filename = input("Enter filename to write/update: ").strip()

```

```

82         if not filename:
83             print("[!] Filename cannot be empty.")
84             continue
85         content = input("Enter content to write: ")
86         response = send_request("WRITE", filename, content)
87         print(f"\n[Server Status] : {response.get('status')}")
88         print(f"[Server Message]: {response.get('message')}")
89
90     # 4. DELETE File
91     elif choice == '4':
92         filename = input("Enter filename to delete: ").strip()
93         if not filename:
94             print("[!] Filename cannot be empty.")
95             continue
96         response = send_request("DELETE", filename)
97         print(f"\n[Server Status] : {response.get('status')}")
98         print(f"[Server Message]: {response.get('message')}")
99
100    # 5. LIST Files
101    elif choice == '5':
102        response = send_request("LIST")
103        print(f"\n[Server Status] : {response.get('status')}")
104        print(f"[Server Message]: {response.get('message')}")
105        if response.get("status") == "SUCCESS":
106            files = response.get("data", [])
107            print("-----")
108            print("FILES ON DFS SERVER:")
109            if files:
110                for idx, f in enumerate(files, 1):
111                    print(f"  {idx}. {f}")
112            else:
113                print("  (No files available)")
114            print("-----")
115
116    # 6. EXIT
117    elif choice == '6':
118        print("\n[+] Exiting DFS Client. Goodbye!")
119        break
120
121    else:
122        print("\n[!] Invalid choice. Please select an option between 1 and
        ↪ 6.")
123
124    if __name__ == "__main__":
125        main()

```

Output

```
=====
      DISTRIBUTED FILE SYSTEM (DFS) CLIENT
=====

1. Create File
2. Read File
3. Write / Update File
4. Delete File
5. List Files
6. Exit

=====

Enter option (1-6): 1
Enter filename to create (e.g., test.txt): test.txt
Enter initial file content (optional): echo "Hello!"

[Server Status] : SUCCESS
[Server Message]: File 'test.txt' created successfully.

=====
      DISTRIBUTED FILE SYSTEM (DFS) CLIENT
=====

1. Create File
2. Read File
3. Write / Update File
4. Delete File
5. List Files
6. Exit

=====

Enter option (1-6): 2
Enter filename to read: test.txt

[Server Status] : SUCCESS
[Server Message]: File 'test.txt' read successfully.

-----
FILE CONTENT:
echo "Hello!"
-----

=====
      DISTRIBUTED FILE SYSTEM (DFS) CLIENT
=====

1. Create File
2. Read File
3. Write / Update File
4. Delete File
5. List Files
6. Exit
```

```

=====
Enter option (1-6): 3
Enter filename to write/update: test.txt
Enter content to write: ok

[Server Status] : SUCCESS
[Server Message]: File 'test.txt' written successfully.

=====
DISTRIBUTED FILE SYSTEM (DFS) CLIENT
=====
1. Create File
2. Read File
3. Write / Update File
4. Delete File
5. List Files
6. Exit
=====
Enter option (1-6): 5

[Server Status] : SUCCESS
[Server Message]: Files listed successfully.
-----
FILES ON DFS SERVER:
    1. himal.txt
    2. test.txt
-----

=====
DISTRIBUTED FILE SYSTEM (DFS) CLIENT
=====
1. Create File
2. Read File
3. Write / Update File
4. Delete File
5. List Files
6. Exit
=====
Enter option (1-6): 4
Enter filename to delete: test.txt

[Server Status] : SUCCESS
[Server Message]: File 'test.txt' deleted successfully.

=====
DISTRIBUTED FILE SYSTEM (DFS) CLIENT
=====
1. Create File

```

```

2. Read File
3. Write / Update File
4. Delete File
5. List Files
6. Exit
=====
Enter option (1-6): 6

[+] Exiting DFS Client. Goodbye!
(venv) PS E:\Lab Reports\Distributed-System-Lab>

```

test.py (test_dfs.py)

Source Code

```

1  import os
2  import sys
3  import time
4  import threading
5  import unittest
6
7  # Import server and client modules
8  sys.path.append(os.path.dirname(os.path.abspath(__file__)))
9  import server
10 import client
11
12 class TestDFS(unittest.TestCase):
13     @classmethod
14     def setUpClass(cls):
15         # Start server in a background daemon thread
16         cls.server_thread = threading.Thread(target=server.start_server,
17         ↪ daemon=True)
18         cls.server_thread.start()
19         time.sleep(0.5) # Give server time to bind and listen
20
21     def test_01_create_file(self):
22         res = client.send_request("CREATE", "lab3_test.txt", "Hello DFS
23         ↪ World!")
24         self.assertEqual(res["status"], "SUCCESS")
25         self.assertIn("created successfully", res["message"])
26
27     def test_02_read_file(self):
28         res = client.send_request("READ", "lab3_test.txt")
29         self.assertEqual(res["status"], "SUCCESS")
30         self.assertEqual(res["data"], "Hello DFS World!")
31
32     def test_03_write_file(self):

```

```

31         res = client.send_request("WRITE", "lab3_test.txt", "Updated content
    ↪   for Distributed File System.")
32         self.assertEqual(res["status"], "SUCCESS")
33
34         # Verify read after write
35         res_read = client.send_request("READ", "lab3_test.txt")
36         self.assertEqual(res_read["data"], "Updated content for Distributed
    ↪   File System.")
37
38     def test_04_list_files(self):
39         res = client.send_request("LIST")
40         self.assertEqual(res["status"], "SUCCESS")
41         self.assertIn("lab3_test.txt", res["data"])
42
43     def test_05_delete_file(self):
44         res = client.send_request("DELETE", "lab3_test.txt")
45         self.assertEqual(res["status"], "SUCCESS")
46
47         # Verify read fails after delete
48         res_read = client.send_request("READ", "lab3_test.txt")
49         self.assertEqual(res_read["status"], "ERROR")
50
51 if __name__ == "__main__":
52     unittest.main()

```

Output

```

=====
Distributed File System (DFS) Server
Running on 127.0.0.1:5000
Storage Path: e:\Lab Reports\Distributed-System-Lab\LAB-3-dfs\server_storage
=====
Waiting for client connections...

.....
-----
Ran 5 tests in 0.524s

OK

```

Discussion

In this experiment, a functional Distributed File System (DFS) was built using Python's fundamental `socket`, `threading`, `json`, and `os` libraries. The client-server model was implemented to allow remote file manipulation over a TCP socket connection.

Key insights from the implementation:

- **TCP Communication Protocol:** The client serializes request parameters (action, filename, content) into JSON strings, which are transmitted reliably over TCP socket connections to the server.
- **Server Concurrency:** Using Python's `threading.Thread`, the server handles incoming client connections concurrently without blocking other request queues.
- **Flat File & Directory Services:** Physical files are safely handled inside `server_storage/`. Filename inputs are sanitized using `os.path.basename` to prevent directory traversal attacks.
- **Transparency:** The client connects directly to the hardcoded server address without prompting the user for manual connection parameters.

Conclusion

The implementation of the Distributed File System in Python was successfully completed and evaluated. All required file system operations (CREATE, READ, WRITE, and DELETE) along with file listing function were verified through manual interactions and automated unit tests. The lab demonstrated fundamental concepts of network sockets, concurrent server handling, and remote file management in distributed systems.